



PHP PAS À PAS

L'objectif de ce tutoriel est de faire une brève présentation des notions de base du langage **PHP**. Il est donc destiné au débutant, mais il pourrait aussi bien servir de références aux programmeurs intermédiaires. Nous ferons entre autre un aperçu sur MySQL et Apache puisque le trio PHP-Apache-MySql va souvent ensemble. Ce tutoriel suppose que vous avez quelques notions de base en informatique, notamment en HTML, et la programmation structurée. Ce qui fait que nous ferons un survol rapide de la syntaxe de base de PHP tout en mettant beaucoup plus l'accent sur les particularités et les fonctions les plus courantes. Comme éditeur de codes sources, vous pouvez aller du simple éditeur NotePad de Windows (mais enregistrez le fichier sous l'extension « .php ») à tout une gamme d'autres éditeurs selon vos préférences (se conférer au site web : <http://cyberzoide.developpez.com/php4/editeurs/>)

Comme l'indique le titre du document : « PHP PAS À PAS », nous présenterons pas à pas différents éléments du langage PHP, ensuite suivront des exemples de codes commentés. Nous ne présenterons que quelques points de l'aspect étudié. D'autres tutoriaux plus détaillés et plus spécifiques suivront après.

Pour plus de détails sur certaines fonctions utilisées dans ce tutoriel, nous vous recommandons le site : <http://www.nexen.net/index.php>

Pour la réalisation de ce tutoriel, nous nous servons du logiciel EasyPHP dont nous expliquerons succinctement le fonctionnement. Si vous constatez une erreur dans ce document, n'hésitez pas à nous la signaler. L'auteur se dégage cependant de toute responsabilité quant à l'utilisation directe ou indirecte de cet article.

SOMMAIRE

- I. PRÉSENTATION**
- II. INSTALLATION DE EASYPHP**
- III. SYNTAXE DE BASE I**
- IV. SYNTAXE DE BASE II**
- V. LES TABLEAUX**
- VI. LES CHAÎNES DE CARACTÈRES**
- VII. LES EXPRESSIONS RÉGULIÈRES**
- VIII. LES VARIABLES PRÉDÉFINIES (ou "super globales")**
- IX. LES FICHIERS**
- X. MANIPULATION DES DATES**
- XI. LES COOKIES**
- XII. LES SESSIONS**
- XIII. PHP ET LES FORMULAIRES**
- XIV. PROGRAMMATION OBJET**
- XV. LE FICHIER HTTPD.CONF**
- XVI. LE FICHIER PHP.INI**
- XVII. PHP ET MYSQL**

PRÉSENTATION

PHP

PHP (acronyme récursif pour PHP Hypertext Preprocessor), a été développé initialement par Rasmus Lerdorf. En 1997, PHP devient un projet collectif et est réécrit par Zeev Suraski et Andi Gutmans. C'est un langage de script qui fonctionne côté serveur. Comme il supporte tous les standards du web et qu'il est gratuit, il s'est rapidement répandu sur la toile. Le langage continue d'évoluer et est actuellement à sa version 5. Ce tutoriel ne s'attarde pas sur une version spécifique.

Site officiel : <http://www.php.net/>

MySQL

C'est un puissant Système de Gestion de Base de Données qui fonctionne sur la plupart des plates-formes. C'est la plus populaire des bases de données open source. Son utilisation est gratuite pour le développement d'application non commerciale. La dernière version stable est la 4.1 qui ne supporte pas les procédures stockées, triggers (déclencheurs), vues, clés étrangères. Les développeurs de MySQL AB travaillent activement sur la série des 5.X qui comblera ses faiblesses mais qui est disponible en version Alpha.

Site officiel : <http://www.mysql.com/>

Apache

C'est le plus populaire des serveurs web depuis Avril 1996. De récents sondages indiquent plus de 68% des sites web l'utilisent. Il est actuellement à la version 2.x

Site officiel : <http://httpd.apache.org/>

EasyPHP

C'est un paquetage pour l'installation et la configuration automatique d'un environnement de travail complet sous Windows, permettant de mettre en oeuvre toute la puissance et la souplesse qu'offre le langage PHP et son support efficace des bases de données. EasyPHP regroupe un serveur Apache, une base de donnée MySQL, le langage PHP ainsi que des outils facilitant le développement de vos sites ou de vos applications. Nous allons l'utiliser pour ce tutoriel car il est utilisé comme plate-forme de développement par un grand nombre de programmeurs.

Site officiel : <http://easyphp.org/?lang=fr>

INSTALLATION DE EASYPHP

Vous pouvez télécharger EasyPHP à l'adresse : <http://easyphp.org/telechargements.php3> , nous utiliserons pour ce tutoriel la version 1.7. Le démarrage du programme d'installation vous présentera un écran sous la forme (**Fig. 1**).

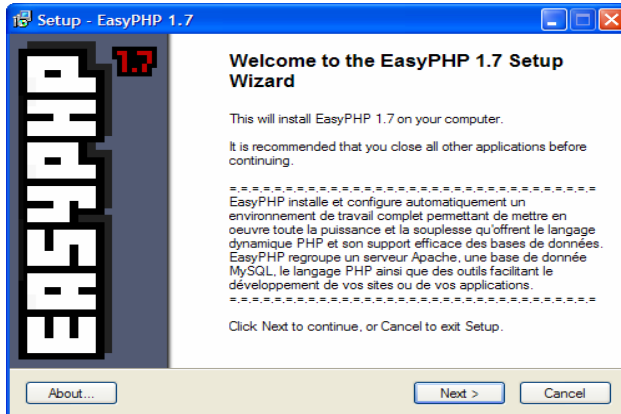


Fig. 1

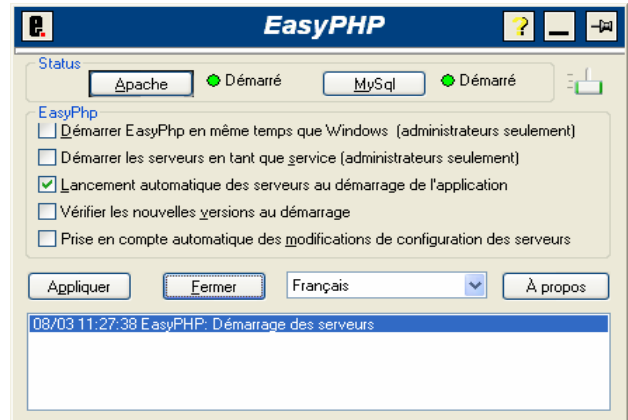


Fig. 2

Nous vous recommandons de garder les options par défaut que le programme d'installation vous propose, ce qui installera l'application dans le répertoire : `C:\Program Files\EasyPHP1-7`. De même, l'interface d'administration sera ajoutée à votre menu « démarrer » de Windows. Veuillez le sélectionner et bien vérifier que les serveurs soient démarrés (**Fig. 2**). Si tel n'est pas le cas, vous pouvez les démarrer, ce qui ajoutera une icône clignotante de EasyPHP à votre barre des tâches.

VÉRIFICATION DE L'INSTALLATION

Par défaut, si vous tapez `http://localhost` dans votre navigateur, les pages web affichées seront celles du répertoire **`C:/Program Files/EasyPHP/www`** qui est la racine du serveur web indiquant donc que ce dernier fonctionne. Vous enregistrerez par la suite vos fichiers dans ce répertoire ou un sous répertoire de ce dernier. Également, par défaut, le lien **`http://localhost/mysql/index.php`** donne accès à l'interface de PhpMyAdmin pour l'administration graphique de la base de données MySQL.

Si tous ceux-ci fonctionnent, tapez dans votre éditeur de choix le code ci-dessous et enregistrez-le sous le nom : « `infos.php` » dans le répertoire `www`. En y accédant via l'URL `http://localhost/infos.php`, ceci vous affichera les informations de configuration de PHP.

```
<?php  
    phpinfo();  
?>
```

SYNTAXE DE BASE I

Un script PHP débute par : `<?php` ou `<?>`. Mais, pour raison de portabilité, `<?php` est le plus conseillé. Il se termine par `'?>'`. Ainsi, l'allure typique d'un code PHP sera `<?php votre code PHP ?>`, et ce code pourra être inscrit dans du code HTML et le tout enregistré au format .php.

I. FONCTIONS DE SORTIE

Ces fonctions sont principalement utilisées pour l'affichage d'informations. Il s'agit de « echo » et « printf » et peuvent prendre des balises de formatage HTML.

Code1	Code2	Résultats
<pre><html> <head> <title>Titre</title> </head> <body> <?php echo "Hello World !"; ?> </body> </html></pre>	<pre><html> <head> <title>Titre</title> </head> <body> <?php print ("Mon nom est Komi SODOKE"); ?> </body> </html></pre>	Code1 : Hello World ! Code2 : Mon nom est Komi SODOKE

II. LES VARIABLES

Une variable doit être précédée d'un dollar (\$). Attention à bien respecter la casse car ces deux variables sont différentes : \$x et \$X. Le nom de la variable doit impérativement commencer par une lettre mais les autres caractères peuvent ensuite contenir des chiffres et même le caractère souligné (_). Mais contrairement aux langages tel le C/C++ ou le Java, PHP est faiblement typé, ce qui signifie que vous n'êtes pas obligé de déclarer le type d'une variable avant de l'utiliser.

III. LES OPÉRATEURS

Arithmétique : Il s'agit de : l'addition (+), la multiplication(*), la soustraction(-), la division (/) et le modulo(%).

Logiques : Il s'agit de : NON (!), ET (&&), OU inclusif (||), OU exclusif (xor).

Relationnels : strictement inférieur (<), inférieur ou égal (<=), strictement supérieur (>), supérieur ou égal (>=).

Incrémentation : Par exemple **ces expressions** : \$i = \$i + 1 ou \$i += 1 ou \$i++ ou ++\$i.

Décrémentation : Par exemple **ces expressions** : \$i = \$i - 1 ou \$i -= 1 ou \$i-- ou --\$i.

SYNTAXE DE BASE II

COMMENTAIRE

Un commentaire qui tient sur une seule ligne peut s'écrire comme suit :

```
//ceci est un commentaire ou #ceci est aussi un commentaire
```

Pour écrire un commentaire de plusieurs lignes: « /* » pour ouvrir et « */ » pour fermer.

```
/*  
ceci est un commentaire multi lignes  
*/
```

CONDITIONNELLE : SI ... ALORS ... SINON ...

```
<?php  
    $myName="komi";  
    if($myName=="xxxx") echo"<b>Correct</b>";  
    else echo"<b>Incorrect , </b> Mon nom est : $myName";  
?  
>
```

LA BOUCLE CONDITIONNELLE : WHILE

```
<?php  
$i = 0;  
while ($i < 5) { echo "itération ".$i."<br>"; $i++; }  
?  
>
```

LA BOUCLE INCONDITIONNELLE : FOR

```
<?php  
  
    for ($i=0; $i<5; $i++)  
    {  
        echo "itération ".$i."<br>";  
    }  
?  
>
```

FONCTIONS DE VARIABLES

`isset (mixed var, mixed var, ... )` permet de savoir si une variable est définie ou non.
`unset (mixed var, mixed var, mixed ... )` détruit la ou les variable passée(s) en argument.
`print_r (mixed expression, bool return)` affiche la structure simple d'une variable.
`var_dump (mixed expression, mixed expression, ... )` affiche la structure complexe d'une variable.

```
<?php  
$var1=1;  
$var2="Variable2";  
  
if(isset($var1))print("\$var1 existe.<br>");else print("\$var1 n'existe pas");  
print_r($var1); echo"<br>";  
var_dump($var2); echo"<br>";  
unset ($var1);  
if (!isset($var1)) print ("\$var1 n'existe plus.<br>");  
?  
>
```


LES TABLEAUX

En PHP, les tableaux représentent une collection de données hétérogènes identifiées chacun par une clé arbitraire qui peut être un entier ou une chaîne de caractères. Donc il est possible d'avoir un tableau contenant des réels, des entiers, des chaînes de caractères et même d'autres tableaux.

CREATION DE TABLEAU

```
$tab1=array(); //Création d'un tableau vide
```

```
array(cle1=>valeur1, cle2=>valeur2, ...) //Pour un tableau contenant des données.
```

Exemple:

```
$tab2=array(      1=>"Element1",
                  10=>"Element2",
                  5=>"5",
                  'nom'=>"Komi SODOKE",
                  'poids'=>"75,9 kg",
                  "M");
var_dump($tab2);
```

L'accès à un élément se fait grâce à l'opérateur []. Par exemple : `echo $tab2['nom'];`

QUELQUES FONCTIONS

count(mixed var, int mode) : affiche le nombre d'éléments du tableau var. Le paramètre optionnel mode est disponible depuis PHP 4.2.0.

```
$nb=count ($tab2);
echo"$nb"; //Affichera 6
```

sort (array array, int sort_flags) : permet de trier un tableau en ordre croissant. Cette fonction assigne de nouvelles clés aux éléments de array. La fonction **rsort**(array array, int sort_flags) permet de faire un tri en ordre décroissant plutôt. Le paramètre optionnel sort_flags peut être utilisé pour modifier le comportement de tri.

```
$tab3=array("cde", "bcd", "def", "abc");
var_dump($tab3);
sort($tab3);      echo"<br>";
var_dump($tab3); //Affichera tab3, trié en ordre croissant
```

asort(array array, int sort_flags) : permet de trier un tableau en ordre croissant en gardant les associativités.

Array_rand(array input , int num_req) : retourne la clé de n éléments du tableau choisis aléatoirement.

implode (string glue, array pieces) : permet de rassembler les éléments d'un tableau « pieces » en une chaîne séparés par les caractères « glue ».

Il existe d'autres fonctions dont voici l'utilisation dans le code ci-dessous.

```
<?php

$tab4=array(10,20,50,80);
$tab5=array(20,30,80,50,100);

//fusionner les 2 tableaux en 1
$mergeArray=array_merge($tab4,$tab5);

//trouver l'intersection des 2 tableaux
$intersectArray=array_intersect($tab4,$tab5);

//Trouver la différence des deux tableaux
$diffArray=array_diff($tab4,$tab5);

?>
```

LES CHAÎNES DE CARACTÈRES

En PHP, on désigne par `string` une suite de caractères alphanumériques. Vous n'avez pas besoin d'une déclaration explicite pour signaler une chaîne de caractère. Ainsi, `$var1= "abcde"`, représente une chaîne de caractères.

QUELQUES FONCTIONS

strlen(`string str`) : calcule la taille (nombre de caractère) d'une chaîne. Permet de concaténer deux chaînes (`$chaîne.$chaîne2`).

strcmp(`string str1` , `string str2`) : permet de faire une comparaison lexicographique (comme celui utilisé pour trier les mots du dictionnaire) des chaînes de caractères. Retourne 0, si les deux chaînes de caractères comparées sont identiques. **strcasecmp** permet d'ignorer la casse.

substr(`string string` , `int start` , `int length`) : permet de couper une chaîne de caractères

str_replace(`mixed search` , `mixed replace` , `mixed subject` , `int count`) : Remplace toutes les occurrences de « `search` » par « `replace` » dans « `subject` ».

strstr(`string haystack` , `string needle`) : permet de vérifier l'existence d'une sous chaîne dans une autre chaîne.

strtolower (`string str`) : pour convertir tous les caractères alphabétiques d'une chaîne en minuscule. Et **strtoupper**(`string str`) : pour la conversion en majuscule.

implode(`string glue` , `array pieces`) : retourne une chaîne contenant la représentation en chaîne de caractères de tous les éléments du tableau pièces , dans le même ordre, avec la chaîne glue , placée entre deux éléments.

trim(`string str` , `string charlist`) : Supprime les espaces (ou d'autres caractères) en début et fin de chaîne.

```
<?php
$str1="exemple1 string1";
$str2="exemple2 string2";
echo"La longueur de la '$str1' est ".strlen($str1)."<br>";
echo"Les 6 premiers caractères de '$str2' sont : ".
substr($str2,0,6)."<br>";
echo"La chaîne '$str1' au majuscule donne sera " .
strtoupper($str1)."<br>";
if (strstr($str1,"str")) echo"La chaîne '$str1' contient la sous chaine
: 'str' <br>";

if (strcmp($str1,$str2)!=0) echo"Les deux chaîne ne sont pas
identique.<br>";

if( strcmp($str1,str_replace($str2,"2","1")) ) echo"Les deux chaîne sont
maintenant identique.<br>";
?>
```

LES EXPRESSIONS RÉGULIÈRES

Les expressions régulières (en abrégé regexps : regular expression) est un des meilleurs outils de manipulation (reconnaissance, validation et extraction suivant un pattern) des chaînes de caractères. En effet, vous pouvez vouloir savoir si une chaîne de caractère correspond à un email valide (c'est-à-dire qu'il est sous la forme : vous@votredomaine.com) ou un code postale valide etc. Pour son fonctionnement les regexps définissent des méta-caractères pour la reconnaissance des patrons à savoir :

méta-caractères	caractères
\d	Nombres 0-9
\D	Tout ce qui n'est pas un nombre
\w	Tout caractère alphanumérique ou soulignement (_)
\W	Tout sauf les caractères alphanumériques ou (_)
\s	Les caractères d'échappement (espace, tabulation, retour chariot)
\S	Tout sauf les caractères d'échappement
.	Tous caractères autre q'un retour chariot

Pour spécifier le nombre de fois que le match se produit, il faudrait utiliser :

?: 0 ou 1 fois; ***** : 0 ou n fois; **+** : une fois ou plus; **{, n }**: au plus n fois; **{m,}**: m fois ou plus; **{m,n}** m à n fois

preg_match(string pattern, string subject, array matches, int flags, int offset) permet de savoir si une chaîne de caractères contient un patron particulier.

preg_replace(mixed pattern, mixed replacement, mixed subject, int limit) : permet de faire des substitutions dans une chaîne de caractères selon un patron donné.

```
<?php
$str1=" Mon Code postale est E1A-2X4 ici au Canada.";
$pattern1 = '/ \w{1}\d{1}\w{1}-\d{1}\w{1}\d{1} /';
if( preg_match($pattern1,$str1) ) echo"La chaine '$str1' contient un code postal valide.";

$site="http://www.devafrique.com/index.html";
if ( preg_match("/^(http:\\/\\/)?([^\./]+)/i",$site, $matches) );
{
    $host = $matches[2];
    echo"<br> '$site' : est une adresse de site web valide.";
}
// repérer les deux derniers segments du nom de l'hôte
preg_match("/[^\.\./]+\.[^\.\./]+$/", $host, $matches);
echo "<br>Le nom de domaine est : ".$matches[0]."\n";
?>
```

LES VARIABLES PRÉDÉFINIES (ou "super globales")

Elles sont une nouveauté du PHP 4.2.0. De façon générale, une variable super globale est disponible dans tous les environnements d'exécution. Elle a donc une portée globale.

\$_SERVER : Contient les variables fournies par le serveur web.

```
<?php
//le nom relatif du serveur qui exécute le script
$server_name=$_SERVER['SERVER_NAME'];
//le répertoire racine du serveur
$document_root=$_SERVER['DOCUMENT_ROOT'];
//le nom du fichier du script courant, depuis la racine
$php_self=$_SERVER['PHP_SELF'];

//l'adresse IP du client qui demande la page
$remote_addr=$_SERVER['REMOTE_ADDR'];

$http_user_agent=$_SERVER['HTTP_USER_AGENT']; //le navigateur client

echo "$server name,      $document root,      $php self,      $remote addr,
$http_user_agent<br>";
?>
```

\$_POST :

Couvre l'ensemble des champs renseignés par la méthode "POST" d'un formulaire. La valeur du champ "name" d'un formulaire est accessible dans les variables \$_POST['name'] .

\$_GET :

Couvre l'ensemble des champs renseignés par la méthode "GET" d'un formulaire ou dans l'URL. La valeur du champ "name" envoyées par GET est : \$_GET['name']

CONSTANTES PRÉDÉFINIES

Il en existe un grand nombre dont voici quelques unes :

```
<?php
//le chemin complet de la page en cours
echo "Le fichier courant est : ".__FILE__."<br>";

//numéro de la ligne de la constante
echo "la ligne courante est : " .__LINE__."<br>";

//la version de PHP
echo "la version de l'analyseur PHP est : ", PHP_VERSION."<br>";

//le système d'exploitation du serveur
echo "le système d'exploitation du serveur est ", PHP_OS."<br>";
?>
```

LES FICHIERS

1. OUVERTURE DE FICHIER

Ceci se fait avec la fonction `fopen`. Le fichier peut être ouvert selon plusieurs modes à savoir:

- « r »: ouvre le fichier en lecture seulement; positionne le pointeur au début du fichier.
- « w »: ouvre le fichier en écriture seulement; positionne le pointeur au début du fichier. Si le fichier n'existe pas, il est créé.
- « a »: ouvre le fichier en écriture seulement; positionne le pointeur à la fin du fichier. Si le fichier n'existait pas, il est créé.
- « x » crée et ouvre un fichier; positionne le pointeur au début du fichier. Échec si le fichier existe.

On pourrait ajouter l'option "+" à ces modes pour spécifier la lecture/écriture.

2. ÉCRITURE DANS UN FICHIER

Ceci se fait avec la fonction `fwrite(resource handle, string string, int length)` après ouverture du fichier en écriture.

3. LECTURE DE FICHIER

Ceci peut se faire avec la fonction `fread(resource handle, int length)` après l'ouverture du fichier en lecture jusqu'à la fin du fichier identifiée par EOF (End Of File). La fonction `fgets(resource handle, int length)` permet de lire ligne par ligne.

4. INFORMATIONS SUR LE FICHIER

Ceci peut se faire avec la fonction `fstat(resource handle)` qui retourne un tableau contenant des informations statistiques (la taille du fichier, la date de modification etc.) sur le fichier.

5. MANIPULATION DE FICHIERS

copy(string source, string dest) : pour copier un fichier de la source à une nouvelle destination.

unlink(string filename, resource context): Pour supprimer un fichier.

rename(string oldname, string newname, resource context): Pour déplacer un fichier de la source à une nouvelle destination.

```
<?php
$fp1=fopen("fichier1.txt","a");
if(!$fp1) { die("Impossible de créer le fichier !!!"); }

for($i=0; $i<10; $i++) fwrite($fp1,"Ligne ".$i."\r\n"); fclose($fp1);

$fp2=fopen("fichier1.txt","r");
while(!feof($fp2)) { $line=fgets($fp2); echo"$line<br>"; }

echo"les informations sur ce fichier : <br>".print_r(fstat($fp2))."<br>";
fclose($fp2);

if(!copy("fichier1.txt","fichier2.txt")) echo"Impossible de copier.";
if(!unlink("fichier1.txt")) echo"<br>Impossible de supprimer.<br>";
?>
```

MANIPULATION DES DATES

PHP conserve en interne la date sous forme de timestamp UNIX, c'est-à-dire le nombre de secondes écoulées depuis le début de l'époque UNIX, (1er janvier 1970 00:00:00 GMT).

1. RECUPERER LA DATE

`time()` : permet de récupérer le timestamp UNIX actuel.

`getdate()` : permet de récupérer un tableau contenant les informations sur le timestamp passé en paramètre. Par défaut prend le timestamp courant.

2. FORMATAGE DES DATES

PHP propose deux fonctions pour le formatage des dates à savoir : `date()` et `strftime()`. Chacune de ces fonctions prend certains paramètres dont ceux cités plus bas. Mais contrairement à la fonction `date()` qui ne présente que les dates en Anglais, il est possible de spécifier la localisation avant l'utilisation de `strftime()` ce qui aura pour effet d'afficher la date dans la langue correspondante.

Date()	Strftime()	Description
d	%d	Le jour du mois entre 01 et 31
F	%B	Le nom entier du mois (Janvier à Décembre)
l	%A	Le jour de la semaine : (Dimanche à Samedi)
H	%H	Heure courante en format 24h: 00-23
i	%M	Minutes correspondant : 00 à 59

3. CREATION DE DATE

Il est possible de récupérer le timestamp correspondant à une date. Ceci se fait grâce à la fonction **mktime** dont tous les arguments sont optionnels.

En utilisant le timestamp et en considérant qu'un jour correspond à 24 heures (soit $24 \times 60 \times 60$ secondes), il devient facile à faire des opérations d'addition et de soustraction sur les dates.

```
<?php
$time=time();
echo"Le timestamp UNIX actuel est : ".$time."<br>";
$tomorrow=getdate($time+24*60*60); //Récupérer la date de demain
print r($tomorrow);

echo"La date actuelle est : ".date('l d F H:i', $time)."<br>";
//Afficher la date en Français Canadien
$locale=setlocale(LC_TIME,"french-canadian");
echo"La date actuelle est : ".strftime('%A %d %B\t %H:%M', $time)."<br>";

$var=mktime(19,20,0,05,02,1981);//Entrer ici une date dont vous voulez
connaitre le jour. heure,min,sec,mois,jour,annee
echo"La jour correspondant au '24 mars 1979' est : ".strftime('%A',
$var)."<br>";
?>
```

LES COOKIES

Un cookie est un petit fichier texte placé dans l'ordinateur de l'internaute à la demande du serveur lors de la visite d'une page. Ce fichier permet de stocker un petit nombre d'informations, comme un mot de passe, le nombre de visites, etc. Les cookies doivent être utilisés avec attention car un internaute peut les supprimer manuellement ou configurer son navigateur pour les refuser.

1. CRÉATION DE COOKIES

Ceci se fait avec la fonction `setcookie`. Elle prend pour paramètres :

\$name : le nom du cookie (cette chaîne de caractères ne doit contenir ni virgule, ni espace, ni deux points (:)).

\$value : la valeur du cookie.

\$expires : la date d'expiration du cookie au format timestamp Unix (c'est-à-dire le nombre de secondes écoulées depuis le 1er janvier 1970), s'il n'est pas spécifié, le cookie sera supprimé lors de la fermeture du navigateur.

\$path : le répertoire du site web pour lequel le cookie est valide (les sous répertoires sont acceptés) : par défaut le répertoire actuel.

\$domain : le nom de domaine pour lequel le cookie est valide.

\$secure : indique (0 ou 1) si le cookie doit être transmis via une connexion sécurisée par le protocole HTTPS.

Tous les paramètres sont optionnels sauf `$name`.

```
<?php
setcookie("Visites", "1", time()+24*60*60, "/", "devafrique.com", 0);
?>
```

Cet exemple envoie le cookie nommé **Visites** de valeur **1**, de date d'expiration : le jour suivant (aujourd'hui + 24 heures). Il est valide dans tout le site web (c'est-à-dire depuis la racine / du site) pour le domaine devafrique.com. Il ne requiert pas de connexion sécurisée par HTTPS.

2. LECTURE DE COOKIES

La puissance du PHP fait qu'il n'est pas nécessaire d'aller chercher quoi que ce soit : PHP le fait automatiquement. Tous les cookies valides pour le domaine et le répertoire du script PHP en cours vont être mis en mémoire. Ainsi vont être créées des variables portant le nom des cookies et valant leurs valeurs respectives.

```
<?php
    echo $_COOKIE['Visites'];    // affiche "1"
?>
```

3. SUPPRESSION DE COOKIES

Pour supprimer un cookie, c'est très simple, il suffit de le recréer à l'identique mais sans valeur et avec une date d'expiration dépassée.

```
<?php
    setcookie("Visites", "", time()-100, "/", " devafrique.com", 0);
```


LES SESSIONS

Une session est en quelque sorte l'équivalent du cookie avec quelques différences. En effet, une session est un fichier stocké à l'inverse du cookie non pas sur le disque du visiteur mais sur le serveur. Une session est personnel à chaque visiteur (un fichier par visiteur), et permet comme le cookie de stocker temporairement des variables de type primitif (car elles seront automatiquement sérialisées lors de l'écriture pour la sauvegarde et désérialisées lors du chargement). Les variables de "type primitif" sont les variables chaînes, entières, flottantes, caractères, tableaux, ... mais en aucun cas les variables ressources comme celles qui sont utilisées pour les bases de données, créer des images ... La durée d'une session est définie par l'hébergeur mais on peut parfois modifier s'il l'autorise la durée par défaut de conservation de la session car elle est automatiquement supprimée en général au bout de 30 minutes.

1. DEBUTER UNE SESSION

Le démarrage explicite et manuel de la session se fait en utilisant la fonction `session_start()`.

2. SAUVEGARDER UNE VARIABLE

Ce fait avec la fonction `session_register()` et permet de sauvegarder des variables de types primitifs. Sous PHP>4.1.0 ceci peut se faire de la façon suivante :

```
$_SESSION['variable']=$variable;
```

3. RECUPERATION DE DONNEES

A partir PHP>4.1.0, on récupère le contenu d'une variable de session en utilisant le tableau associatif suivant : `$_SESSION["nom_de_votre_variable"]`. Pour savoir si une variable appartient à une session, vous pouvez utiliser la fonction `session_is_registered(variable)`. Cependant, il est aujourd'hui conseillé d'utiliser la fonction `isset()` qui prend comme argument la variable à tester.

4. SUPPRIMER UNE VARIABLE D'UNE SESSION

Ceci se fait avec la fonction `session_unregister(string name)`, name étant le nom de la variable à détruire. `session_unset()` permet de détruire toutes les variables de session.

La fonction `session_destroy()` qui ne prend aucun paramètre et permet de détruire une session.

```
<?php
session_start(); //Création de la session
$id=session_id();
echo"L'id de la session est : $id<br>";

$nom = "Mon nom";
//Sauvegarde dans la session créée de la variable $nom
if(session_register("nom")) echo "Variable $nom est sauvegardée.<br>";
else echo "Erreur : la variable $nom n'a pas pu être sauvegardé.<br>";

//Sauvegarde dans la session créée de la variable nom2, PHP>4.1.0
$_SESSION["nom2"]=$nom. "2";

if(session_unregister("nom")) echo"Variable $nom supprimée avec succès.";
else echo "La suppression de la variable $nom a échoué. <br>";

session_unset(); //Pour effacer toutes les variables de la session

if(session_destroy())echo "Session détruite !<br>";
```

```
else echo "Erreur : impossible de détruire la session !<br>";  
?>
```

PHP ET LES FORMULAIRES

Il est assez simple de récupérer dans une variable PHP le contenu d'un champ d'un formulaire. Cependant, il existe un cas un peu spécial : ce sont les cases à cocher car dans ce cas là, il faut vérifier l'existence de la variable, mais aussi utiliser un tableau pour stocker les options qui ont été cochées par l'utilisateur. Nous allons, en guise d'exemple, utiliser le formulaire très simple ci-dessous :

```
<form action="manipForm.php" method="post" name="formDemo">
champ Text <input name="champText" type="text" size="10" maxlength="10"><br>
Champ Mot de passe<input name="pwd" type="password" size="10"
maxlength="10"><br><br>
<input name="champCache" type="hidden" value="1">

radio_option1 <input name="radio_option" type="radio" value="1"><br>
radio_option2 <input name="radio_option" type="radio" value="2"><br>
radio_option3 <input name="radio_option" type="radio" value="3"><br><br><br>

check_option1 <input name="check_option[]" type="checkbox" value="1"><br>
check_option2 <input name="check_option[]" type="checkbox" value="2"><br>
check_option3 <input name="check_option[]" type="checkbox" value="3"><br>

<br><br><input value="Soumettre" type="submit">
</form>
```

Fichier : manipForm.php

```
<?php

/*Recuperation des informations contenue dans le champ texte. Si le champ n'est
pas renseigné, $champText est vide*/

//Recuperation des informations contenue dans le champ cache
$champText=$_POST['champText'];

//Recuperation des informations contenue dans le champ mot de passe
$champCache=$_POST['champCache']; $pwd=$_POST['pwd'];

echo"champText=$champText, champCache=$champCache, pwd=$pwd<br>";

/*Pour les boutons radio, $_POST['radio_option'] n'est definit que si une option
est selectionne.*/
if(isset($_POST['radio_option'])) { $radio_option=$_POST['radio_option'];
echo"radio_option".$radio_option." est selectionne<br>"; }

/*identique a celui des cases radio mais dans ce cas on obtient un tableau
contenant la valeur des élément sélectionné*/
if(isset($_POST['check_option']))
{
    $check_option=$_POST['check_option'];
    foreach($check_option as $checkedValue) echo"check_option".$checkedValue."
est selectionne<br>";
}

//le tableau superglobal $_POST contient tous ce que par le formulaire a envoyé
print_r($_POST);
```

?>

PROGRAMMATION OBJET

Nous supposerons que les concepts liés à la programmation orientée objet sont déjà connus. Seul son implantation en PHP ainsi que les avantages qu'elle peut procurer seront présentés. Il est important de souligner ici que PHP5 apporte d'importants changements en ce qui concerne la programmation objet avec PHP.

1. CREATION D'UNE CLASSE

Création d'une classe très simple, ne contenant qu'un seul attribut (\$myvar), un constructeur, et une méthode (afficher()).

```
class myclass {  
    var $myvar;  
    function myclass($myparameter) { //constructeur  
        $this->myvar = $myparameter;  
    }  
    function afficher() {  
        echo "myvar vaut : $this->myvar";  
    }  
}
```

2. INSTANCIATION ET MANIPULATION DES OBJETS

```
$myobject = new myclass(10);  
$mytab[1] = new myclass(10);  
$myobject->afficher(); //Affichera myvar vaut : 10.
```

3. HÉRITAGE

Nous allons déclarer une classe fille myclass2 qui étend la classe mère myclass. Dans cette classe, nous allons ajouté un attribut myvar2 et une méthode afficher2(\$color) qui ne seront définis que dans cette classe. Nous allons également redéfinir un constructeur myclass2(\$parameter, \$parameter2) qui permet d'initialiser les 2 attributs de l'objet lors de sa création et une fonction afficher2 qui permettra d'afficher les variables avec la couleur \$color.

```
class myclass2 extends myclass {  
    var $myvar2 ;  
    function myclass2($myparameter, $myparameter2) {  
        $this->myvar = $myparameter;  
        $this->myvar2 = $myparameter2;  
    }  
    function afficher2($color) {  
        echo "<font color=\"\$color\">myvar vaut : $this->myvar</font><br>";  
        echo "<font color=\"\$color\">myvar2 vaut : $this->myvar2</font><br>";  
    }  
}  
$myobject2 = new myclass2(10, 20);  
$myobject2->afficher2("red");  
$myobject2->afficher();
```

4. SERIALISATION

serialize() et unserialize() permettent de sauvegarder et de restaurer les objets. PHP dispose de deux fonctions « magiques » : __sleep qui lorsque définit, permet de fermer les variables

ressources ouvert par l'objet avant la sérialisation; __wakeup() au contraire est appelé lors de la restauration de l'objet et permet de restaurer ces ressources.

LE FICHIER HTTPD.CONF

Ce fichier se trouve par défaut dans le répertoire : **C:\Program Files\EasyPHP1-7\apache\conf**. Il s'agit d'un fichier de configuration de apache. Vous pouvez l'éditer à l'aide du bloc-notes. Si vous modifier certaines informations, vous pouvez arrêter et redémarrer le serveur avant qu'elles ne prennent effet. Vous y trouverez beaucoup de directives de configuration dont les suivantes :

ServerAdmin : Cette directive indique l'adresse électronique de l'administrateur du serveur web. Elle apparaîtra lors d'éventuelles erreurs. Vérifiez que cette ligne est bien présente: **ServerAdmin** vous@votredomaine.com

ServerName : Cette directive permet de définir un nom pour votre serveur. Vérifiez que cette ligne est bien présente : **ServerName** localhost ou **ServerName** localhost:80. Vous pourrez accéder à votre serveur en tapant cette adresse dans la barre d'adresse de votre navigateur Internet : http://localhost ou http://127.0.0.1

DocumentRoot : Cette directive spécifie le répertoire racine de votre site. Seules les pages présentes dans ce dossier seront interprétées par le serveur. Vous pouvez le remplacez par une autre valeur par exemple, C:\sitesWeb, dans ce cas vous devez aussi remplacer la ligne :

<Directory "C:\Program Files\EasyPHP1-7\www\" : qui spécifie le répertoire de base du serveur le chemin par le nouveau chemin.

DirectoryIndex indique quels sont les fichiers à envoyer au navigateur lorsque le visiteur tape une adresse se terminant par le nom d'un répertoire. Il s'agit en fait de la page par défaut du répertoire. Vous pouvez y spécifier plusieurs noms de fichiers séparés par des espaces vides. Apache renverra le premier fichier qu'il trouvera dans le répertoire avec une priorité allant de gauche à droite.

```
<IfModule mod_dir.c>
    DirectoryIndex index.html index.php index.htm
</IfModule>
```

Alias renvoie à un autre endroit du disque. Cela permet de stocker des documents ailleurs que dans le répertoire racine et de les référencer. Pour illustrer ceci, déplaçons le dossier de phpmyadmin situé dans le répertoire racine de EasyPhp1-7 vers c:\www\pma

Recherchez la ligne # Alias et ajoutez en dessous Alias /pma "c:/www/PhpMyAdmin"

Options permet de définir les fonctionnalités du serveur. Elle s'utilise avec Directory. Voici les valeurs qu'elle peut prendre :

ExecCGI : L'exécution de scripts CGI est permise.

FollowSymLinks : Le serveur est autorisé à suivre les liens symboliques dans ce répertoire.

Indexes : Si une URL pointant sur un répertoire et qu'aucun fichier défini par DirectoryIndex n'existe dans ce répertoire, alors le serveur retourne une liste formatée du contenu du répertoire.

LE FICHIER PHP.INI

Il s'agit du fichier de configuration de PHP. Vous le trouverez dans le répertoire **C:\Program Files\EasyPHP1-7\apache**. Par défaut, à l'installation la configuration est faite pour le développement et non la production. Donc il faudrait bien connaître le rôle de certaines directives de ce fichier afin d'y apporter les modifications voulues plus tard pour des raisons de sécurité et de productivité. La plupart de ces directives sont précédées d'explications sur leur fonctionnement. Pour commentez une ligne, il suffit de la précéder de « ; ».

short_open_tag = On; Ceci permet de spécifier si vous autorisez l'utilisation de la balise '<?' Comme balise de début de script PHP.

error_reporting = E_ALL; // Définit quel type d'erreur sera affichée.

Par défaut toutes les erreurs et avertissements sont affichés. Cela est recommandé pour le développement. Mais on peut le mettre à `error_reporting = E_ALL & ~E_NOTICE` ; pour afficher seulement les erreurs.

display_errors = On; et **display_startup_errors** = Off ; Ces deux directives contrôlent l'affichage ou non des messages d'erreurs dans le navigateur client.

log_errors = Off ; pour spécifier l'écriture ou non des erreurs dans un fichier de log.

error_log = filename; pour spécifier le chemin dans lequel serait localiser le fichier contenant les erreurs.

Pour la production, il est recommandé de mettre `log_errors = On` et `error_log` doit pointer sur un fichier du nom : **phperrors.log** situé la racine du serveur web.

register_globals = Off ; Nouveau à partir de PHP 4.1.0 cette directive contrôle l'accès aux variables d'environnement. Par défaut est mis à off pour des raisons de sécurité. L'accès à ses variables se fait donc par les tableaux **superglobaux**.

file_uploads = On; Pour spécifier si vous autorisez l'upload sur le serveur via http.

Windows Extensions

Contient une série de nom de fichier dll que vous aurez à activer selon les besoins de vos projets.

Il est possible de modifier la valeur de ces directives dynamiquement dans un programme en utilisant la fonction **ini_set**.

PHP ET MYSQL

PHP est compatible avec un bon nombre de SGBD. Parmi ces derniers, MySQL est sans doute le plus populaire parce qu'il a beaucoup de choses en commun avec PHP : gratuité, disponibilité du code source, etc. Voici les quelques fonctions d'interfaçage de MySQL avec PHP :

mysql_connect() : se connecte au serveur avec le trio classique "serveur", "login", "pwd" en argument.

mysql_error() : retourne le message d'erreur de MySQL.

mysql_select_db() : sélectionne la base passée en argument.

mysql_query() : envoie la requête SQL passée en argument.

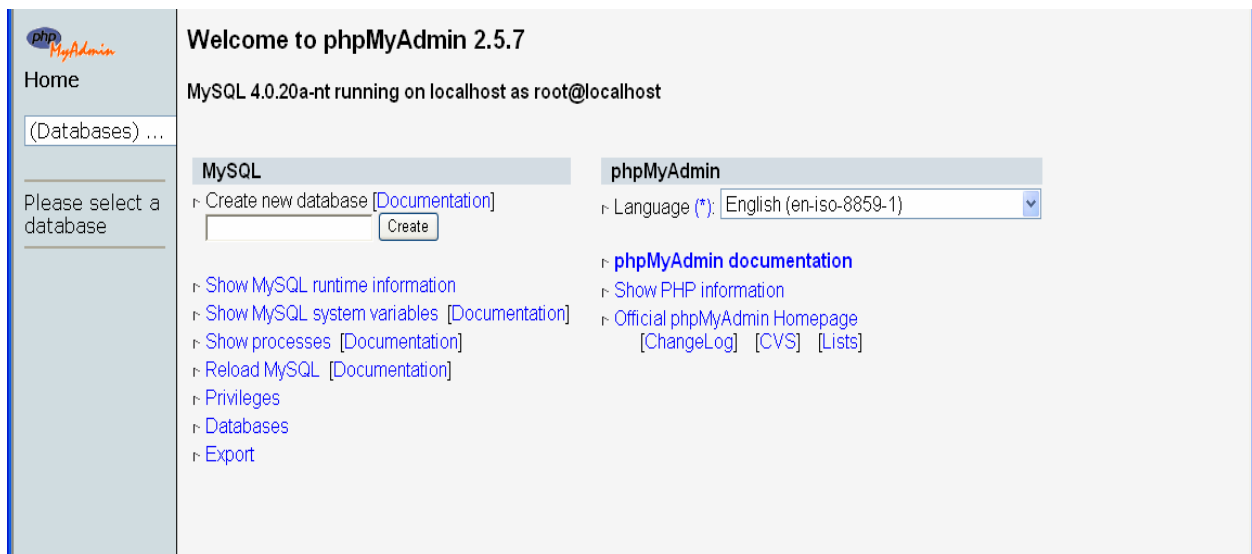
mysql_num_rows() : retourne le nombre de lignes du résultat de la requête passée en argument.

mysql_fetch_array() : retourne une ligne du résultat de la requête passée en argument sous forme d'un tableau associatif.

mysql_free_result() : libère la mémoire allouée pour la requête passée en argument.

mysql_close() : ferme la connexion passée en argument.

Lorsque vous tapez l'adresse : localhost/mysql/index.php, phpMyAdmin se charge et vous présentera une interface qui ressemble à l'écran ci-dessous :



Créez une base de données « db1 ».

Cela vous présentera ensuite une interface ressemblant à : (pour vous permettre d'ajouter une table.)

Database *db1* running on *localhost*

Database *db1* has been created.

SQL-query : [\[Edit\]](#) [\[Create PHP Code\]](#)
`CREATE DATABASE 'db1' ;`

[Structure](#) [SQL](#) [Export](#) [Search](#) [Query](#) [Drop](#)

No tables found in database.

• Create new table on database *db1* :
Name :
Fields : [Go](#)

Créez une table de nom « table1 » ayant 3 champs : nom, age, email.

Database *db1* - Table *table1* running on *localhost*

Field	Type [Documentation]	Length/Values*	Attributes	Null	Default**	Extra	Primary	Index	Ur
nom	VARCHAR	15		null			☐	☐	☐
age	TINYINT		UNSIGNED	not null			☐	☐	☐
email	VARCHAR	20		not null			☐	☐	☐

Table comments :

Table type : [Default](#)

[Save](#)

* If field type is "enum" or "set", please enter the values using this format: 'a','b','c' ...
If you ever need to put a backslash ("\") or a single quote ("'") amongst those values, backslashes it (for example "\\xyz" or 'a\'b').

** For default values, please enter just a single value, without backslash escaping or quotes, using this format: a

[\[Documentation\]](#)

Le code suivant permettra d'ajouter 3 enregistrements dans la table, ensuite de les afficher.

À suivre...